

return, Multiple Inputs, and Function Behavior

Day 4 Morning - Functions and Program
Design

30 min teaching + 30 min exercises

```
def calculate_total(price,  
quantity):  
    return price * quantity  
  
result = calculate_total(10, 5)
```

```
def classify_error_count(error_count):  
    if error_count == 0:  
        return "valid"  
    if error_count <= 5:  
        return "warning"  
    return "rejected"
```

Goal: write functions that give useful results back to the program.

BLOCK 26

Where this block fits

Block 25 created functions. Block 26 focuses on what functions give back.



Instruction focus

- `print()` versus `return`
- numbers, strings, booleans
- collections and multiple results
- early returns
- default and keyword arguments

Practice focus

Students write predictable functions that can be called, stored, compared, and tested.

BLOCK 26

The big distinction

`print()` communicates with a human. `return` communicates with code.

`print()`

Show something on the screen.

Useful for messages, reports, and debugging.

`return`

Send a value back to the caller.

Useful for calculations, decisions, and composition.

A value printed is visible. A value returned is usable.

This is one of the most important function concepts in beginner Python.

BLOCK 26

When print() is not enough

This function displays 50, but gives nothing useful back to the caller.

```
def calculate_total(price, quantity):  
    print(price * quantity)  
  
result = calculate_total(10, 5)  
  
print(result)
```

Observed output

50

None

Why?

The function printed the answer, but did not return it.

So result becomes None.

Beginner warning: seeing output does not mean your function returned a value.

BLOCK 26

The reusable version returns the value

Now the caller receives the result and can decide what to do with it.

```
def calculate_total(price, quantity):  
    return price * quantity  
  
result = calculate_total(10, 5)  
  
print(result)
```

calculate



return 50

store in result



print later

Design habit

Let calculation functions return values. Print at the edge of the program.

BLOCK 26

Returned values can keep working

A returned value can be stored, compared, printed, or passed onward.

Store

```
total = calculate_total(10, 5)
```

Compare

```
if total > 100:
```

Print

```
print(f"Total: {total}")
```

Pass onward

```
tax = calculate_tax(total, 0.07)
```

Add to collection

```
totals.append(total)
```

return makes a function part of a larger program, not just a display tool.

BLOCK 26

Functions can return different kinds of values

The caller should know what kind of result to expect.

```
def calculate_total(price,  
    quantity):  
    return price * quantity
```

Number

```
def classify_status(errors):  
    if errors == 0:  
        return "valid"  
    return "warning"
```

String

```
def is_valid_score(score):  
    return 0 <= score <= 100
```

Boolean

Good functions are predictable: the name, inputs, and return type all make sense together.

BLOCK 26

Returning Boolean values directly

Avoid unnecessary if/else when the comparison already produces True or False.

```
def is_valid_score(score):  
    if 0 <= score <= 100:  
        return True  
    else:  
        return False
```



```
def is_valid_score(score):  
    return 0 <= score <= 100
```

Both are correct

The shorter version says the same thing more directly.

BLOCK 26

Functions can return lists

Returning a collection lets the caller keep processing the results.

```
def get_invalid_values(values):  
    invalid_values = []  
  
    for value in values:  
        if value < 0:  
            invalid_values.append(value)  
  
    return invalid_values
```

```
numbers = [5, -2, 9, -1]  
invalid = get_invalid_values(numbers)  
  
print(invalid)
```

Output
[-2, -1]

The function does not print each bad value. It returns the list for the caller to use.

BLOCK 26

Functions can return dictionaries

Dictionaries are useful when a function creates a structured result.

```
def create_summary(total, errors):  
    return {  
        "total_records": total,  
        "error_count": errors,  
    }
```

```
summary = create_summary(500, 3)  
  
print(summary["total_records"])  
print(summary["error_count"])
```

Design idea

Return a dictionary when the result has named parts that belong together.

BLOCK 26

Multiple return values

Python can return more than one value at a time.

```
def get_minimum_and_maximum(values):  
    minimum = min(values)  
    maximum = max(values)  
  
    return minimum, maximum
```

```
values = [8, 3, 10, 4]  
  
minimum, maximum =  
    get_minimum_and_maximum(values)  
  
print(minimum)  
print(maximum)
```

Mental model

Python is returning a tuple-like package, and the caller unpacks it into two names.

BLOCK 26

Early returns

A function can return before reaching the bottom of its body.

```
def calculate_percentage(completed, total):  
    if total <= 0:  
        return None  
  
    return completed / total * 100
```

invalid total?

return None

otherwise

return %

Why it helps

Validation functions often read more clearly when invalid cases return early.

BLOCK 26

Early returns in validation

Each rejection is handled as soon as it is discovered.

```
def validate_record_id(record_id):  
    record_id = record_id.strip()  
  
    if record_id == "":  
        return False  
  
    if not record_id.startswith("EQ-"):  
        return False  
  
    return True
```

Read it as a checklist

1. Empty ID? reject.
2. Wrong prefix? reject.
3. Otherwise valid.

Benefit

The final return True means every earlier rejection was avoided.

BLOCK 26

Default arguments

A function can provide a normal value when the caller does not specify one.

```
def calculate_processing_time(record_count,  
rate=250):  
    return record_count / rate
```

```
time = calculate_processing_time(10000)  
# uses rate=250  
  
time = calculate_processing_time(10000,  
500)  
# overrides rate
```

Use when

One value is common enough to be the normal case, but should still be changeable.

BLOCK 26

Keyword arguments

Naming the arguments can make a call easier to read.

```
def calculate_processing_time(record_count,  
rate=250):  
    return record_count / rate
```

```
time = calculate_processing_time(  
    record_count=10000,  
    rate=500  
)
```

Why it helps

When several numbers appear in a call, names prevent mystery values.

Keyword arguments trade a little extra typing for clarity.

BLOCK 26

Design predictable functions

The caller should not be surprised by what the function does.

Predictable inputs

The function name and parameters show what information is needed.

Predictable output

The return value has a clear type and meaning.

Avoid surprise behavior

Do not print from a calculation function unless display is the function's job.

Handle invalid cases intentionally

Return a clear result, or reject invalid input with a clear policy.

A good function is boring in the best possible way.

BLOCK 26

Guided exercise setup

Students write functions and verify what each one returns.

Practice rule

For every function, ask:

What inputs does it need?

What value does it return?

What invalid values matter?

Time plan

Exercise 1: 4 min

Exercise 2: 5 min

Exercise 3: 6 min

Exercise 4: 6 min

Exercise 5: 6 min

Challenge: remaining time

The goal is not just “it ran.” The goal is “I know what it returned.”

BLOCK 26

Exercise 1: print() or return?

Classify each function by whether it should display or give back a value.

```
def show_welcome_message():  
    print("Welcome")  
  
def calculate_discount(price):  
    return price * 0.10  
  
def format_status(status):  
    return status.strip().lower()
```

Task

For each function, decide whether print() or return is appropriate. Explain why.

Time box
4 minutes

Hint: Is the function's job display, or is its result needed later?

BLOCK 26

Exercise 2: validation function

Create a Boolean function for completion values.

```
def validate_completion(completion):  
    # Return True when completion is valid.  
    # Return False otherwise.  
    pass  
  
print(validate_completion(0))  
print(validate_completion(50))  
print(validate_completion(100))  
print(validate_completion(101))
```

Rules

Valid completion must be:

- at least 0
- no greater than 100

Time box

5 minutes

Stretch: test -1 and 100.1 after the basic cases work.

BLOCK 26

Exercise 3: classification function

Return a string that describes the error level.

```
def classify_error_count(error_count):  
    # 0 -> "valid"  
    # 1 to 5 -> "warning"  
    # more than 5 -> "rejected"  
    pass  
  
print(classify_error_count(0))  
print(classify_error_count(1))  
print(classify_error_count(5))  
print(classify_error_count(6))
```

Task

Use if / elif / else or early returns.
The function should return a string,
not print it.

Time box

6 minutes

Boundary values matter: 0, 1, 5, and 6 should all be tested.

BLOCK 26

Exercise 4: create a summary dictionary

Return one structured object from several inputs.

```
def create_record_summary(record_id, status,
error_count):
    return {
        "record_id": record_id,
        "status": status,
        "error_count": error_count,
    }

summary = create_record_summary("EQ-1001", "ready",
0)
print(summary)
```

Task

Create the function, then access each dictionary field after the call.

Time box
6 minutes

Question: why return a dictionary instead of printing three separate lines?

BLOCK 26

Exercise 5: multiple results

Return minimum, maximum, and average from one function.

```
def calculate_statistics(values):  
    minimum = min(values)  
    maximum = max(values)  
    average = sum(values) / len(values)  
  
    return minimum, maximum, average  
  
low, high, avg = calculate_statistics([8, 3, 10, 4])  
print(low, high, avg)
```

Task

Type the function, then test at least two different lists.

Time box

6 minutes

Stretch: what should happen if the list is empty?

BLOCK 26

Applied challenge: validate one record

Return two values: a status and a message.

```
def validate_record(record):  
    if record["record_id"] == "":  
        return "rejected", "Record ID is missing"  
  
    if record["error_count"] > 5:  
        return "rejected", "Too many errors"  
  
    if record["error_count"] > 0:  
        return "warning", "Minor errors detected"  
  
    return "ready", "Record passed validation"
```

Task

Create several record dictionaries. Call `validate_record()` for each one and unpack the result.

```
status, message =  
validate_record(record)  
print(status)  
print(message)
```

Time box

Remaining practice time

BLOCK 26

Block 26 wrap-up

Students should now design functions by their inputs and outputs.

Students can explain

- print() versus return
- why returned values are reusable
- Boolean-returning functions
- returning lists and dictionaries
- early returns

Students can do

- write multi-input functions
- return strings, numbers, booleans
- return multiple values
- use default arguments
- call functions with keyword arguments

Next block: scope, side effects, and designing small functions with clear responsibilities.